

Objective

This lab gives students hands-on practice with **materialized views** built over two simple tables. Students create single-table, join, filtered, time-based, and derived-value materialized views; observe how inserts, updates, and deletes make stored results stale; refresh them with a **COMPLETE** refresh; and then use **materialized view logs** to explore **FAST** refresh, including the rules that decide which definitions can and cannot be fast-refreshed.

Case Study: University Academic Records

Background.

The academic records office of a university keeps its data in two flat tables: **Students**, with one row per student (degree program, degree title, section, and enrollment date), and **Course_Results**, with one row per course result (marks, grade points, credit hours, and attendance). The accompanying load script generates about 100 students and 600 results across Fall 2022 to Spring 2026. Every semester new results are posted, and management keeps asking for the same reports — program-wise performance, section summaries, low-attendance lists, grade distributions, and semester trends — each of which means re-joining and re-aggregating both tables. The office therefore wants these reports stored as materialized views.

- 1 Repeated academic reports.** Course-wise, program-wise, section-wise, and semester-wise summaries are requested every term, some needing a join between the two tables.
- 2 Changing base data.** Results are posted continuously and student records are occasionally corrected. Stored results become stale until they are refreshed.
- 3 Choosing a refresh strategy.** A **COMPLETE** refresh rebuilds everything; **FAST** refresh propagates only the changes — but not every definition qualifies. The team must know which is which.

Database Tables

Use the two tables below. The accompanying schema-and-data-load script creates them and loads the sample data. **Course_Results.StudentID** refers to **Students.StudentID** (one student, many results); this is the column you will join on.

STUDENTS

Column	Type	Description
StudentID	NUMBER (PK)	Unique student identifier
StudentName	VARCHAR2(100)	Student name
DegreeProgram	VARCHAR2(10)	BS, MS, or PhD
DegreeTitle	VARCHAR2(50)	Data Science, Computer Science, or Software Engineering
Section	VARCHAR2(20)	Intake section, e.g. Fall22 ... Fall26

EnrollmentDate	DATE	Date of enrollment
----------------	------	--------------------

COURSE_RESULTS

Column	Type	Description
ResultID	NUMBER (PK)	Unique result identifier
StudentID	NUMBER	Refers to Students.StudentID
CourseName	VARCHAR2(60)	e.g. Database Systems, Data Warehousing
Semester	VARCHAR2(20)	e.g. Fall23, Spring24
Marks	NUMBER(5,2)	Total marks out of 100
GradePoints	NUMBER(3,2)	0.00 to 4.00
CreditHours	NUMBER	2, 3, or 4
AttendancePct	NUMBER(5,2)	Attendance percentage

Sample rows (illustrative — the 'script generates the actual

StudentID	StudentName	DegreeProgram	DegreeTitle	Section	EnrollmentDate
1001	Student 001	BS	Data Science	Fall23	14-AUG-2023
1002	Student 002	MS	Computer Science	Fall24	05-AUG-2024
1003	Student 003	BS	Software Engineering	Fall22	21-AUG-2022

ResultID	StudentID	CourseName	Semester	Marks	GradePoints	CreditHours	AttendancePct
5001	1001	Database Systems	Spring24	78.50	3.33	3	91.25
5002	1002	Data Warehousing	Spring25	88.00	4.00	3	72.40
5003	1003	Statistics	Fall23	52.75	1.00	2	68.10

Note. For new rows use **StudentID** ≥ 2001 and **ResultID** ≥ 9001 so they do not clash with the generated data. Prefix your materialized views with **MV_** so you can find them easily in Task 15.

Tasks

Part A — Single-Table Materialized Views

Aggregates over Course_Results alone; no join needed.

- 1 Create a materialized view that stores, for each **CourseName**, the **average marks**, **average grade points**, and **total number of results**.
- 2 Create a materialized view that stores, for each **Semester**, the total number of results and the **total credit hours** (a time-based summary).
- 3 Create a materialized view that stores the **highest and lowest marks** for each **CourseName**.

Part B — Join, Filtered, Time-Based & Derived Materialized Views

Combine Students with Course_Results, grouped by student attributes.

- 4 Create a materialized view that joins Students and Course_Results and stores, for each **DegreeProgram** and **DegreeTitle**, the average grade points and the number of results.
- 5 Create a materialized view that stores, for each **Section**, the number of results with **attendance below 75%** and their average marks (use a WHERE clause).
- 6 **Time + join.** Create a materialized view that stores, for each **enrollment year** (extracted from EnrollmentDate) and **Semester**, the number of results and the average grade points.
- 7 Create a materialized view that derives a **letter grade** from Marks using CASE ($A \geq 85$, $B \geq 70$, $C \geq 60$, $D \geq 50$, otherwise F) and stores the **grade distribution** — the number of results per DegreeTitle and letter grade.

Part C — Staleness & COMPLETE Refresh

Change the base tables and see which stored results are no longer correct.

- 8 Insert at least **3 new students** and **10 new results**, spread across two semesters including **Fall26**. Compare the materialized view from Task 4 with the same aggregate run directly on the base tables, and check the **STALENESS** column of **USER_MVIEWS** for all your materialized views.
- 9 Perform a **COMPLETE refresh** of the materialized views from Tasks 4 and 5 (DBMS_MVIEW.REFRESH) and confirm that the new data is reflected. Record the **LAST_REFRESH_DATE**.
- 10 Now **update the Section** of one student who already has results. Without refreshing, compare **every** materialized view with its defining query. Which stored results still match and which do not? Explain using the columns each view depends on, and compare the **STALENESS flag** with what you actually observe in the data.

Part D — Materialized View Logs & FAST Refresh

Propagate only the changed rows, and learn where FAST refresh breaks down.

11

Create a **materialized view log** on `Course_Results` with the information needed to FAST-refresh aggregate and join materialized views (`ROWID`, `SEQUENCE`, `INCLUDING NEW VALUES`). Plan ahead: include every `Course_Results` column that the views in Tasks 12 and 14 will use.

12

Create a materialized view on `Course_Results` with **REFRESH FAST ON DEMAND** that stores, per **Semester**, `COUNT(*)`, `SUM(Marks)`, and `COUNT(Marks)`. Insert at least **10 more results**, perform a **FAST refresh**, verify the new rows, and then compute the average marks per semester from the stored columns.

13

Attempt to create a FAST-refresh materialized view that stores **only AVG(Marks)** per Semester. Record the error Oracle raises (or the reason reported by `DBMS_MVIEW.EXPLAIN_MVIEW`, after running `utlxmlv.sql`) and explain why it fails while the view in Task 12 works.

14

FAST refresh with a join. Create a materialized view that lists `StudentID`, `StudentName`, `DegreeProgram`, `CourseName`, and `GradePoints` for every result, using **REFRESH FAST ON DEMAND** (include the `ROWID` of both tables in the `SELECT` list). First attempt it with **no log on Students** and record the error; then create that log, create the view, insert one new result, FAST-refresh, and verify.

15

Create a materialized view with **REFRESH FAST ON COMMIT** that stores per-Semester result counts. Insert one result and `COMMIT`, then query the view **without** any manual refresh. Compare with Task 12 and discuss the cost of refreshing on every commit.